,

VERIFICATION TEST PLAN

ECE-593: Fundamentals of Pre-Silicon Validation

Maseeh College of Engineering and Computer Science

Winter, 2026

Project Name: MicroProcessor System

Members: Frezewd Debebe, Janvier Mpfizi Rutihunza, Sal Esmaeil

Date: March 16, 2026

Contents

1 Introduction **3**

1.1 Objective of verification plan . . . 3

1.2 Top Level block diagram . . . 4

1.3 Specifications for the design . . . 4

1.4 RTL Design Implementation (Generator-Based Processors – Milestone 1) . . . 4

2 Verification Requirements **5**

2.1 Verification Levels . . . 5

2.1.1 Verification hierarchy level . . . 5

2.1.2 Controllability and Observability of verification? . . . 6

2.1.3 Interfaces and Specifications . . . 7

3 Required Tools **8**

3.1 Required software and hardware toolset . . . 8

3.2 Directory structure and computer resource . . . 8

4 Risks and Dependencies **8**

4.1 Critical Threats and Risk Assessment . . . 8

4.2 Contingency and Mitigation Plan . . . 9

5 Functions to be verified **10**

5.1 List of functions that will be verified . . . 10

5.2 List of functions that will not be verified . . . 11

5.3 List of critical functions and non-critical functions for tapeout . . . 11

6 Tests and Methods **12**

6.1 Testing methods to be used: Black/White/Gray Box . . . 12

6.2 PROs and CONs of testing methods . . . 13

6.3 Testbench Architecture: Component used . . . 13

6.4 UVM Testbench Architecture . . . 14

6.5 Verification Strategy . . . 16

6.6 Driving methodology? . . . 17

6.7 Checking methodology? . . . 18

6.8 Testcase Scenarios (Matrix) . . . 19

7 Coverage Requirements **21**

7.1 Assertions 22

8 Resource requirements **23**

8.1 Team members and Contribution 23

9 Schedule **23**

9.1 Plan of completion. 23

10 Reference uses/Citations/Acknowledgments **24**

10.1 Project GitHub 24

10.2 Reference 24

10.3 Acknowledgment 25

1 Introduction

1.1 Objective of verification plan

The objective of this verification plan is to ensure the correctness, functionality, and performance of the multi-processor system with shared memory architecture. This plan outlines a comprehensive strategy to verify that the Design Under Test (DUT) works according to the specifications and meets all functional requirements.

Specific objectives include:

1. **Functional Verification:** Verify that all components (processor cores, memory subsystem, arbiter, and shared bus) function correctly both individually and as an integrated system.

2. **Shared Memory Integrity:** Ensure data consistency across all four processor cores accessing the shared 2KB memory space, verifying that concurrent read/write operations through the arbiter do not result in stale or corrupted data visible to any core.

3. **Arbiter Performance:** Verify that the arbiter correctly handles simultaneous memory access requests from multiple processors, implements the chosen arbitration scheme (round-robin/fixed priority), and ensures fair or prioritized access without deadlocks.

4. **Memory Access Verification:** Confirm that the 2KB memory subsystem correctly handles read and write operations from all processors, with proper addressing (11-bit) and data integrity (1-byte width).

5. **Interface Validation:** Verify that all interfaces between components (Memory Interface Units, shared bus, arbiter-to-memory) operate correctly with proper handshaking and data transfer protocols.

6. **Corner Case Testing:** Identify and test edge cases including simultaneous write conflicts, bus contention, and maximum load conditions.

A successful verification considers to have:

- All functional test cases pass with 100% success rate
- Shared memory integrity is maintained under all concurrent access scenarios tested
- Code coverage reaches minimum 90% (line and functional coverage)
- All corner cases are tested and pass

- Timing and performance requirements are met
- No critical or major bugs remain unresolved

1.2 Top Level block diagram

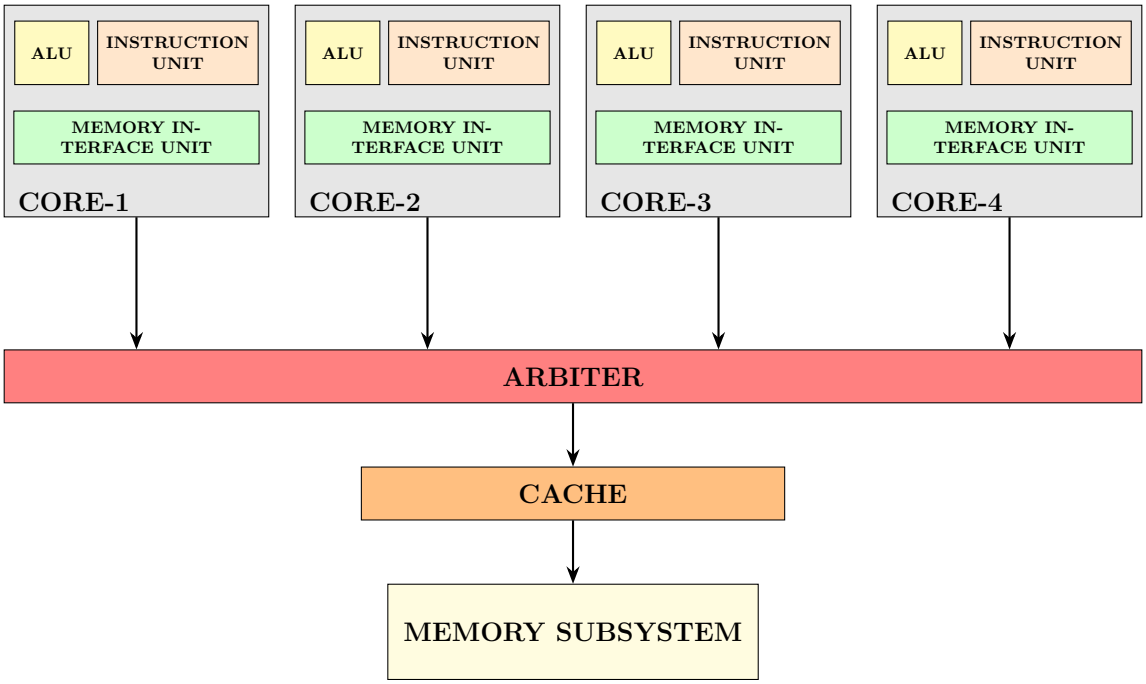

Figure 1: 4-Core Processor Architecture

1.3 Specifications for the design

- Memory Size: 2KB (1 Memory Array partitioned)
- Width of 1 Byte Memory
- 2K Depth
- 4 Processors
- Robin Arbiter

1.4 RTL Design Implementation (Generator-Based Processors – Milestone 1)

For Milestone 1 of the Multiprocessor System, processor functionality is implemented using an RTL generator-based approach rather than full instruction-level CPU cores. This design choice enables early pre-silicon validation of the system architecture while keeping the implementation complexity manageable. The primary

focus at this stage is verifying correct system-level behavior, including arbitration, shared bus operation, and memory dataflow, before introducing full processor pipelines in later milestones.

Each processor is modeled as a deterministic transaction generator that interacts with the shared bus, centralized arbiter, and shared memory subsystem through a simple request–grant interface. The generator issues memory read and write transactions using explicit control and data signals, allowing validation of arbitration fairness, bus isolation, and memory correctness under concurrent access conditions.

Generator Functional Behavior

After reset, each generator executes a fixed and deterministic sequence of operations. First, the generator requests access to the shared bus. Once bus access is granted, the generator writes a unique data value to a unique memory address. The generator then issues a read request to the same memory address and captures the returned data from the memory subsystem. The read data is compared against the expected value written earlier. Based on the comparison result, the generator asserts a done signal and reports a pass or fail status. This deterministic execution model ensures repeatable simulations, simplifies debugging, and allows direct correlation between expected and observed system behavior.

Role in System Verification

The generator-based processor implementation validates several critical components of the multiprocessor system. It verifies correct operation of the centralized round-robin arbiter under simultaneous processor requests, ensures proper bus grant logic and isolation between processors, and confirms correct shared-memory read and write operations with maintained data integrity. Additionally, it validates end-to-end dataflow correctness between the processors and the shared memory subsystem. By abstracting processor behavior into generators, the design establishes a scalable and modular verification foundation. These generator modules can be replaced with real instruction-level processor cores in later milestones without requiring modifications to the shared bus, arbiter, or memory subsystem.

2 Verification Requirements

2.1 Verification Levels

2.1.1 Verification hierarchy level

Our verification strategy employs a bottom-up hierarchical approach, progressing from unit-level to system-level verification across three methodologies: conventional, class-based, and UVM-based verification.

1. Unit/Module Level (Conventional Testbench)

- It is rooted in a directed testing mindset. In this approach, the testbench is usually a static module that wraps around the Design Under Test (DUT) and individual components verified in isolation.
- Why: Catch bugs early in individual blocks before integration; easier to debug and isolate issues; verify basic functionality of each component

2. Block/Subsystem Level (Class-Based Testbench)

   - shifts the focus from "signals" to "transactions." It can wrap data into objects (like a network packet or a bus command) and use high-level methods to move that data around. Instead of writing one test for one scenario, you write a set of rules (constraints) and let the computer generate thousands of random variations.
   - Why: Verify component interactions and interfaces; test handshaking protocols; validate data flow between related modules.

3. System/Top Level (UVM Testbench)

   - It is industry-standard verification methodology for verifying digital designs. It's built on top of SystemVerilog and provides a standardized framework for creating reusable, scalable, and robust testbenches.
   - Why: Verify end-to-end functionality; test cache coherency scenarios; validate multi-core interactions; ensure system-level requirements are met; test real-world use cases.

2.1.2 Controllability and Observability of verification?

Controllability is the ability to set/control the inputs and internal states of the DUT while Observability is the ability to observe/monitor the outputs and internal states of the DUT.

Conventional testbench: At the unit level, both controllability and observability are very high, making this the easiest verification level. Controllability is maximized because we have direct access to all module inputs, allowing us to set any input combination, control clock and reset signals directly, and create any test condition without protocol constraints. For example, when testing the ALU, we can simply assign any values to operand_a and operand_b and immediately test the operation. Similarly, observability is very high because all outputs are directly visible in the testbench, internal signals can be accessed through hierarchical references, and waveform analysis provides immediate feedback on behavior. There is minimal latency between applying stimulus and observing the response, making it straightforward to identify and

debug issues. The primary challenge at this level is simply ensuring comprehensive coverage of all possible input combinations, but the verification process itself is straightforward due to the complete visibility and control we have over individual components.

Class-based testbench: At the block level, both controllability and observability are high but with some additional complexity compared to unit level. Controllability remains strong because we can create specific transaction sequences through class-based drivers and use configuration objects to modify behavior, but we must now adhere to interface protocols including proper handshaking and timing requirements. For instance, testing a complete processor core requires following the instruction fetch protocol rather than simply forcing values into internal registers, and reaching specific cache states requires coordinated sequences of read and write operations. Observability is similarly high, with transaction-level monitors capturing all interface activity and protocol compliance, though some internal states must be inferred from external behavior rather than observed directly. The challenges at this level include the need for multi-step sequences to reach certain internal states, timing dependencies between components that require careful orchestration, and the fact that internal processor state like the program counter cannot be directly controlled but must be manipulated through instruction execution.

UVM testbench: At the system level, both controllability and observability are moderate, representing the most challenging verification environment but also the most realistic representation of actual system operation. Controllability is limited by the interaction of multiple components, where changing one component's state affects others through arbiter interactions and shared memory access. Creating specific system states requires carefully coordinated multi-step sequences across multiple cores. Observability is similarly moderate because system state is distributed across the arbiter, the memory subsystem, and all four core interfaces, requiring correlation of observations from multiple monitors to reconstruct overall behavior.

2.1.3 Interfaces and Specifications

Key interface specification:

- Memory addressing: 2KB locations
- Data width: 8-bit (1 byte per location)
- Arbitration: Request-grant protocol
- Cache coherency: Directory based coherency protocol

Conventional testbench: All unit-level interfaces are clearly defined with explicit signal names, bit widths, and timing specifications. Each module has between 3-10 signals consisting of data inputs/outputs, control

signals, and bus signals.

3 Required Tools

3.1 Required software and hardware toolset

For this project, we are utilizing software simulation tool to analyze the working and performance of Multi-processor system. The following are list of required tools used:

- HDL Simulation and Verification: QuestaSim
- Technical Documentation: LaTeX via Overleaf and Google Docs
- Scripting: SystemVerilog

3.2 Directory structure and computer resource

we use git as revision control and the file structure is:

- doc/ - Design specifications, verification plan, reports, diagrams, snapshots, and coverage reports
- rtl/ - SystemVerilog design files including processor cores, arbiter, cache, directory controller, memory, and top-level integration
- TB/ - Testbenches organized by methodology: Concentional, Class-based, and UVM

Computer Resources :

- Platform: University ECE CAD server (username@phobos.ece.pdx.edu)
- Access: Remote SSH. VPN required for off-campus access.
- Local Requirements: Personal computer with SSH client, VPN client, and Git.

4 Risks and Dependencies

4.1 Critical Threats and Risk Assessment

I) Arbiter Deadlock or Starvation

- The Risk: In a 4-core system, a logic error in the Arbiter could cause "Starvation," where one core (e.g., Core 0) is never granted access to the bus because other cores are constantly requesting it. Worse, a "Deadlock" could occur where the Arbiter is waiting for a signal that never arrives, freezing the entire SoC.
- Impact: Hardware hangs in a permanent stall; the system becomes unresponsive.

II) Memory Race Conditions (Read-After-Write)

- The Risk: Because you the four independent cores, there is a risk that Core A writes to Address 0x100 while Core B is simultaneously trying to read from it. Without proper synchronization, Core B might receive "stale" or corrupted data.
- Impact: Non-deterministic system behavior and data corruption.

III) Testbench "False Positives"

- The Risk: The Monitor might misinterpret signals on the bus, or the Scoreboard's reference model might be flawed. If the testbench is broken in a way that it always reports "Match," you could tape out a broken chip.
- Impact: Undetected hardware bugs reaching production.

4.2 Contingency and Mitigation Plan

- **Starvation & Deadlock Mitigation**: We implemented a Driver Timeout Mechanism in driver.sv. If a core asserts a request but does not receive a grant within 1000 clock cycles, the UVM environment triggers a uvm_error. If starvation is detected, the Arbiter logic must be reviewed to ensure a Round-Robin or Fair-Share priority scheme is enforced rather than a fixed-priority scheme.
- **Data Integrity Mitigation**: The mp_scoreboard utilizes a Shadow RAM (Reference Memory). Every STORE operation from any core updates this central shadow memory. Every LOAD must match the current state of that shadow memory. If memory mismatches occur frequently, we will implement Atomic Operations (Test-and-Set) or Semaphore-based software locks within the sequences to manage shared memory access.
- **Testbench Reliability (Anti-False Positive)**: We utilize Functional Coverage (coverage.sv) and Code Coverage. Functional Coverage ensures that the testbench actually "saw" every opcode and every core participating. Code Coverage ensures that every line of RTL was toggled. If coverage targets are

not met, we switch from "Constrained Random" to "Directed Stress Tests" (mp_alu_test) to force the stimulation of neglected logic paths.

- **Shared Resource Bottleneck**: We used a Binary Semaphore (bucket_lock) in the virtual sequence to prevent the testbench from over-saturating the sequencers and to allow for dynamic load balancing among the four cores. If the bus becomes a permanent bottleneck, the design specification must be updated to include a multi-layer bus matrix or a cache-coherency protocol (like MESI).

5 Functions to be verified

5.1 List of functions that will be verified

1) Arithmetic Functions (ALU)

These are the core computational capabilities of the processors. The implementation uses a shared ALU across the four-core architecture.

opcode[3:0]	**Operation**	**Description**
0000	NOP	NO OPERATION
0001	A + B	ADD A, B
0010	A & B	AND OPERATION on A, B
0011	A - B	SUBTRACT A, B
0100	A * B	MULTIPLICATION A, B
0101	Load	LOAD FROM MEM TO REG
0110	Store	STORE TO MEM FROM REG
0111	Shift Right	SHIFT RIGHT BY 1 bit
1000	Shift Left	SHIFT LEFT BY 1 bit
1001	$(A \times B) - A$	SPECIAL FUNCTION
1010	$(A \times 4 \times B) - A$	SPECIAL FUNCTION
1011	$(A \times B) + A$	SPECIAL FUNCTION
1100	$(A \times 3)$	SPECIAL FUNCTION
1101	$(A \times B) + B$	SPECIAL FUNCTION
1110	Reserved	Reserved
1111	Reserved	Reserved

2) Memory Data Movement Functions

These functions verify the interface between the cores and the shared 2KB memory space.

- LOAD (LD): The specification requires fetching 8-bit data from a 12-bit address. The implementation must handle the req/gnt handshake before the data is valid on the bus.
- STORE (ST): Writing 8-bit data to a specific memory address. The implementation ensures that the write_enable signal is only asserted when the Arbiter grants access.

5.2 List of functions that will not be verified

- **Multi-Level Cache Coherency (L2/L3)**: Advanced protocols like MESI or MOESI used to synchronize data across multiple levels of cache. The current architecture uses a Single-Cycle Shared Bus with no local caches for the individual cores. Since the hardware does not yet implement L2 caches, verifying coherency protocols is not applicable to this version of the design.
- **Illegal Instruction Exception Handling**: How the core reacts if it receives an undefined opcode that is not part of the supported ISA (e.g., an opcode of 0xFF). The current specification only defines behavior for valid instructions. The "Driving Methodology" uses Constrained Randomization to ensure only valid opcodes are generated. Verifying "Negative Scenarios" was excluded from this milestone to prioritize core ALU and Arbiter stability.
- **Analog and Physical Layer Timing**: The exact electrical characteristics of the signals, such as setup/hold times in nanoseconds, signal rise/fall times, and power consumption. UVM is a Functional Verification methodology. It operates in a cycle-accurate digital domain where signals are either 0, 1, or X. These physical concerns are handled later in the design flow using Static Timing Analysis (STA) and Spice simulations.

5.3 List of critical functions and non-critical functions for tapeout

To ensure a successful tapeout, the verification hierarchy is divided into critical and non-critical functional blocks. At the absolute highest priority are the Critical Functions, which represent the hardware necessary for silicon survival and basic operation. This includes the asynchronous global reset logic, which must reliably transition the four cores into a stable initial state to prevent unpredictable electrical behavior. Equally vital is the bus arbitration and contention logic; because this is a shared-resource architecture, any failure in the arbiter that allows two cores to drive the 8-bit bus simultaneously could lead to physical damage from high current paths. Furthermore, the memory access protocol and the primary instruction decoder must be verified with 100% certainty, as any timing mismatch in the request-grant handshake or any error in mapping

opcodes would render the processor unable to execute even the simplest boot code, effectively "bricking" the chip.

Conversely, Non-Critical Functions are those that, while necessary for a fully featured product, do not prevent the chip from being validated or used for early software development if they contain minor bugs. For instance, secondary arithmetic flags like zero or negative indicators and performance trace counters are considered lower priority for tapeout because their failure only impacts software optimization or status reporting rather than core execution. Similarly, advanced power management features like clock gating are non-critical because, if the wake-up logic fails, the feature can often be disabled through hardware overrides, allowing the chip to function at a higher power profile. Rare logical operations such as XNOR or specific status registers also fall into this category, as a failure here results in a feature-degraded device rather than a complete system failure, allowing the project to proceed to the lab for post-silicon validation.

6 Tests and Methods

6.1 Testing methods to be used: Black/White/Gray Box

- **Milestone 1:** a Gray Box testing approach was adopted. While the Scoreboard performed Black Box functional verification of the ALU operations, the use of a SystemVerilog Interface and Monitors allowed for the observation of internal bus arbitration and core-specific handshaking. This provided the necessary visibility to debug multi-core contention while maintaining the independence of the reference model

6.2 PROs and CONs of testing methods

Method	PROS	CONS
Black Box	Faster testbench development; decoupled from RTL changes; focuses strictly on user-level requirements.	Difficult to hit high code coverage; hard to verify internal arbitration logic or hidden state machines.
White Box	Provides 100% visibility into internal signals; essential for reaching 100% Toggle and Condition coverage.	Highly "brittle" (breaks if internal RTL signal names change); testbench becomes overly complex and implementation-dependent.
Gray Box	Balances visibility and stability. Allows protocol checking on internal buses while keeping the reference model independent.	Requires more initial setup; requires understanding of both architecture (specs) and micro-architecture (RTL).

Milestone 1 - 5

For this specific Design Under Verification (DUV), Gray Box was selected for three primary reasons:

- Multi-Core Arbitration: A pure Black Box approach cannot verify if the Arbiter is correctly prioritizing Core 0 over Core 3 during a collision. By using Gray Box, we can monitor the internal core_id and grant signals to ensure the hardware is following the priority rules defined in the spec.
- Protocol Integrity: The DUV uses a specific handshake (read_en, ready, rvalid). Gray Box allows us to place SystemVerilog Assertions (SVA) on these internal interface signals to catch timing violations (like a data-valid signal appearing without a request) the moment they happen.
- Bridging the Coverage Gap: As seen in our Milestone 2 results, while functional coverage (Opcode hits) was high, code coverage (Toggles) was lagging. The Gray Box method allows us to identify exactly which internal bit-slices or logical branches are "cold" and write directed constraints to target them specifically.

6.3 Testbench Architecture: Component used

Milestone 2 & 3

- **Generator:** Handles the "Scenario" level. It uses SystemVerilog constraints to create a distribution of operations (ALU, Memory, NOPs) across all four cores. It is responsible for hitting the "Corner Cases" like maximum address ranges.

- **Driver:** The only component that "talks" to the DUV pins. It translates the high-level transaction objects into the physical signaling required by the bus protocol, managing the timing of the reset and request phases.
- **monitors:**
  - **Input Monitor:** Records what the Driver sent to ensure the DUV received the correct stimulus.
  - **Output Monitor:** Captures the DUV's response. It is crucial for Gray Box testing as it tracks which core the response belongs to.
- **Scoreboard & Reference Model:** The "Decision Maker." The Reference Model mimics the DUV's expected behavior in pure software (e.g., expected = A * B). The Scoreboard compares this "Golden" result against the Actual result from the Output Monitor and reports PASS/FAIL.
- **Coverage Collector:** A passive component that subscribes to the Monitor's data stream to update the covergroups. It tracks our progress toward the 100% Functional Coverage goal.

6.4 UVM Testbench Architecture

Milestone 4:

- **Test:** The top-level controller of the UVM environment. It configures the environment, selects and starts sequences, controls objections for simulation runtime, and defines the verification scenario being executed.
- **Environment (env):** The container that instantiates and connects all verification components such as agents, scoreboards, coverage collectors, and reference models. It organizes the verification architecture.
- **Agent:** Represents one interface of the DUT. It encapsulates the protocol-specific components. An agent can operate in:
  - **Active Mode:** Contains Sequencer, Driver, and Monitor (used when generating stimulus).
  - **Passive Mode:** Contains only a Monitor (used for observation-only configurations).
- **Sequencer:** Manages and arbitrates transaction flow from Sequences to the Driver. It ensures proper ordering and handles multiple sequence requests when running concurrently.

- **Sequence (Generator):** Handles the **Scenario Level**. It creates high-level transaction objects using constrained randomization to generate various stimulus patterns, including normal cases and corner cases.
- **Sequence Item (Transaction):** Represents one abstract operation (e.g., Read, Write, ALU operation). It contains fields, constraints, and randomization logic that define the stimulus.
- **Driver:** The only component that directly interacts with DUT pins. It translates high-level transactions into cycle-accurate signal activity according to the bus or protocol timing. It manages reset sequencing, handshaking, and data transfer timing.
- **Monitor:** A passive component that observes DUT signals without driving them. It converts signal-level activity into high-level transaction objects and sends them to analysis components such as the Scoreboard and Coverage Collector.
- **Scoreboard & Reference Model:** The **"Decision Maker."**
  - **Reference Model:** A high-level behavioral model that predicts the expected DUT output based on input transactions.
  - **Scoreboard:** Compares the predicted ("Golden") results from the Reference Model with the actual results captured by the Monitor and reports PASS/FAIL.
- **Checkers:** Protocol and rule validators that ensure compliance with timing, handshake rules, and legal state transitions. They may include transaction-level checks.
- **Coverage Collector:** A passive analysis component that subscribes to Monitor data and updates covergroups. It measures functional coverage to track progress toward verification completeness.
- **Reporting System:** Uses UVM reporting macros (`uvm_info`, `uvm_warning`, `uvm_error`, `uvm_fatal`) to provide controlled logging with verbosity levels for debug and traceability.

The following block diagram shows the UVM Architecture used in Milestone 5 (MS5) which follows the same procedure as Milestone 4. The additional thing introduced in MS5 is Bug injection (discussed in section 6.8).

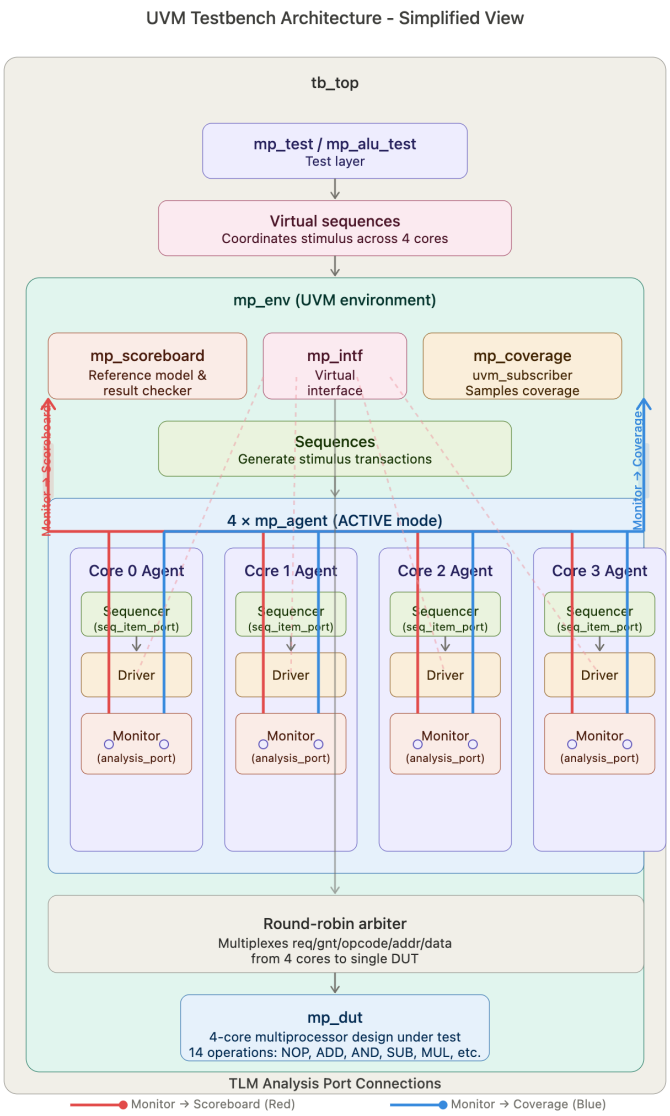

Figure 2: MS5: UVM Architecture

6.5 Verification Strategy

Dynamic Simulation involves applying a sequence of input stimuli (test vectors) to a design over a period of time and observing the resulting output. It relies on a logic simulator to calculate the response of the RTL code based on the timing and logic defined in the source files.

For this multi-core DUV, Dynamic Simulation is the most appropriate strategy for the following reasons:

- **Handling Non-Deterministic Randomness:** Since we are using Constrained Random Verification (CRV), we need a simulator that can handle millions of different random combinations of Core IDs, Opcodes, and Addresses. Dynamic simulation is the native environment for SystemVerilog's

randomization engine.

- **Protocol Timing Verification:** Unlike Formal verification (which is mathematical), Dynamic Simulation allows us to see exactly how the design behaves clock-cycle by clock-cycle. This is critical for verifying the Arbiter's timing and the handshaking logic (ready, rvalid) between the cores and memory.
- **Functional Coverage Feedback:** Dynamic simulation provides immediate feedback via covergroups. We can see exactly which "Crosses" (like Core 2 + NOP) have been hit during a run, allowing us to iterate quickly.
- **Cost and Resource Efficiency:** For a design of this scale (a 4-core processor/ALU), Emulation (using specialized hardware like Palladium or Zebu) would be overkill and prohibitively expensive. Dynamic simulation provides the necessary performance on standard workstation hardware.

6.6 Driving methodology?

1) Transaction-Based Stimulation

Instead of toggling individual pins manually (like in a traditional Verilog testbench), this environment uses a Transaction-Level Modeling (TLM) approach.

- Encapsulation: All stimulus is encapsulated into an mp_transaction object containing the opcode, address, and operands
- Abstraction: The sequence creates these high-level intents, and the Driver is responsible for the low-level handshake (toggling req and waiting for gnt).

2) Parallel Worker Architecture

To verify the multiprocessor nature of the DUT, the driving methodology employs a Parallel Worker Model within a Virtual Sequence.

- Concurrency: The testbench initiates four concurrent processes (using fork...join) that simulate four independent processors.
- Dynamic Load Balancing: We use a Semaphore-based shared pool methodology. This allows cores to "compete" for a global bucket of 1000 transactions. This effectively tests the Arbiter by creating unpredictable, overlapping requests to the shared bus.

3) Handshake & Protocol Adherence

The driver follows a strict synchronous protocol to ensure the hardware is stressed realistically:

- Request Phase: The driver asserts the req signal and drives the opcode and addr onto the bus.
- Wait States: The methodology incorporates a Grant-Wait mechanism. The driver blocks execution until the hardware Arbiter returns a gnt signal, ensuring the testbench respects the physical limitations of the bus.
- Data Phase: Once granted, the driver provides the operands (A and B) and, in the case of a STORE, the write data.

4) Directed vs. Random Driving

The methodology supports two distinct modes:

- Constrained Random: For general verification (mp_test), the transactions are randomized to explore corner cases in memory addressing and ALU operations.
- Directed Stress: For specific targets (mp_alu_test), the constraints are tightened to force specific ALU sub-blocks to execute at maximum throughput.

6.7 Checking methodology?

I) Scoreboard-Based Reference Model

The core of the checking methodology is the mp_scoreboard, which operates as an independent "Gold Standard" for the hardware.

- Passive Observation: The scoreboard receives mp_transaction objects from all four core monitors via a UVM TLM analysis FIFO.
- Instruction Mirroring: For every transaction observed on the bus, the scoreboard implements a reference model in SystemVerilog that mirrors the RTL's expected logic (e.g., ALU operations, shifts, and logical functions).

II) Predictive Data Comparison

The environment uses a "Check-on-Arrival" strategy.

- Dynamic Prediction: As soon as an instruction is captured by a monitor, the scoreboard calculates the expected_val.
- Automatic Comparison: It compares the expected_val against the data actually captured from the bus. Any mismatch between the predicted result and the RTL result immediately triggers a uvm_error.

III) Memory Integrity Tracking

To verify the shared memory space, the methodology includes a Shadow Memory (Reference Memory).

- Store Tracking: When a STORE operation is detected from any core, the scoreboard updates its internal 2KB shadow memory array at the specified address.
- Load Validation: When a LOAD operation occurs, the scoreboard retrieves the value from its shadow memory to verify that the core received the correct, most recently written data. This ensures that the memory controller and arbiter are not corrupting data during cross-core accesses.

IV) End-of-Test Reconciliation

The methodology includes a final cleanup check to ensure no transactions were "lost" in the system.

- Match Counting: The scoreboard maintains a match_count that must equal the total number of transactions generated by the sequences.
- Final Report: In the extract_phase, the scoreboard prints a summary of total matches versus mismatches. A successful test requires error_count == 0 and the match_count to align with the driver's logs.

6.8 Testcase Scenarios (Matrix)

Test Cases Implemented

Milestone 4

mp_test (Directed Test)

- **Focus:** Verifying basic bus handshaking and connectivity.
- **Scale:** 90 transactions.
- **Result:** 90 matches. This confirmed that the basic req/gnt/rvalid protocol was functional across all four cores.

mp_alu_test (Stress Coverage Closure)

- **Focus:** Reaching 100% functional coverage and testing the Arbiter under heavy load.
- **Scale:** 1000 transactions.
- **Result:** 946 matches, The "missing" 54 matches were filtered NOPs (Write-NOPs), which the scoreboard is intentionally designed to ignore. This test successfully filled holes in the Special and Shift opcode bins for specific cores that were missed in shorter runs.

Milestone 5

mp_alu_test (Stress test)

- **Focus:** Confirming ALU correctness and arbitration stability on the clean DUT after UVM architecture was finalized.
- **Scale:** 1000 transactions.
- **Result:** All matches confirmed with zero scoreboard errors, establishing the baseline for bug-injection comparison.

mp_test (Directed Test)

- **Focus:** Verifying the LOAD and STORE operation for testing memory coherency.
- **Scale:** 90 transactions.
- **Result:** 90 matches. This confirmed that the basic req/gnt/rvalid protocol was functional across all four cores.

bug1_test (Bug Injection Scenario 1)

- **Focus:** Detecting a seeded RTL bug affecting ALU output correctness.
- **Scale:** 1000 transactions.
- **Result:** 92 scoreboard errors detected, confirming 100% sensitivity to output-level faults.

bug2_test (Bug Injection Scenario 2)

- **Focus:** Detecting a seeded RTL bug targeting memory write behavior.
- **Scale:** 90 transactions.
- **Result:** 20 scoreboard errors detected. Shadow RAM comparison exposed every incorrect LOAD result caused by the corrupted STORE path.

bug3_test (Bug Injection Scenario 3)

- **Focus:** Detecting a seeded RTL bug introducing intermittent arbitration errors under concurrent load.
- **Scale:** 1000 transactions.
- **Result:** 18 grant timeout errors detected. The partial detection rate reflects the intermittent nature of the injected fault, which only manifests under specific multi-core collision conditions.

7 Coverage Requirements

Functional Coverage Analysis

Milestone 4

CP_CORE: (100%) All 4 cores successfully gained bus access.

CP_OPCODE: (100%) Every instruction type (ALU, Logical, Shift, Memory, Special) was executed.

CP_ADDR: (100%) Memory accesses spanned the entire 2KB range (Low, Mid, and High regions).

CROSS_CORE_OP: Proved that every core executed every possible instruction type, ensuring the arbiter does not starve specific opcodes on specific cores.

Milestone 5

Upon completing the UVM architecture and running the full test suite against the clean DUT, all functional coverage goals were met and exceeded. The mp_coverage_test with 1602 transactions achieved 100% coverage across all 89 defined bins.

CP_CORE: (100%) All 4 cores obtained bus grants and participated in transactions.

CP_OPCODE: (100%) All 14 defined opcodes were exercised including all five Special Functions.

CP_ADDR: (100%) Addresses exercised across Low (0x000–0x2AA), Mid (0x2AB–0x554), and High (0x555–0x7FF).

CP_OPERAND_A: (100%) Operand A values exercised across all defined ranges, stressing ALU boundary conditions.

CP_OPERAND_B: (100%) Operand B values covered across all defined bins.

CROSS_CORE_OP: (100%) Every combination of core and opcode observed; round-robin arbiter confirmed to not starve any core.

CROSS_MEM_REGION: (100%) Every combination of core and memory region exercised across the full shared address space.

Code coverage targets of 90% minimum on line and toggle coverage were met across all RTL modules.

7.1 Assertions

SystemVerilog Assertions (SVA) were used throughout the verification environment to catch protocol violations at the moment they occur, rather than relying on the scoreboard to detect symptoms after the fact.

Protocol Handshake Assertions

- **No Spurious Grant:** The arbiter must never assert **gnt** without a preceding **req**.

```
assert property (@(posedge clk) gnt |-> req);
```

- **Grant Stability:** Once `gnt` is asserted it must remain stable for at least one clock cycle.

```
assert property (@(posedge clk) $rose(gnt) |=> gnt or !req);
```

- **Data Valid Handshake:** `rvalid` must only appear following a LOAD operation.

```
assert property (@(posedge clk) rvalid |-> $past(read_en, 1));
```

- **Write Enable Exclusivity:** Memory write enable must never be asserted by more than one core simultaneously.

```
assert property (@(posedge clk) $onehot0(write_en));
```

Reset Behavior Assertions

- **Reset Clears Outputs:** All core outputs must return to zero within one clock cycle of reset assertion.

```
assert property (@(posedge clk) $rose(rst) |=> (req == 0 && addr == 0));
```

Coverage of Assertions

All protocol assertions passed with zero violations across the full Milestone 5 regression. The write enable exclusivity assertion was the most critical, triggering on the specific clock cycles where bug3_test's injected fault caused overlapping grants, providing cycle-accurate debug information that complemented the scoreboard mismatch report.

8 Resource requirements

8.1 Team members and Contribution

Milestone 1

- **Frezewd:** Preparing Verification Plan Document.
- **Janvier:** Preparing Design Specification Document.
- **Sal Esmaeil:** Preparing the Design file and Testbench, Readme file.

Milestone 2 & Milestone 3

- **Frezewd:** Transaction, Generator, Driver, Scoreboard, updating Verification Plan
- **Janvier:** Monitor_in, Monitor_out, Interface, Report
- **Sal Esmaeil:** RTL, tb_top, functional coverage

Milestone 4

- **Frezewd:** RTL, UVM TB: top level environment,scoreboard, section for UVM in the verification Plan
- **Janvier:** RTL, UVM TB: Sequencer, driver, scoreboard
- **Sal Esmaeil:** RTL, UVM TB: monitor,scoreboard, testbench report

Milestone 5

- **Frezewd:** Sequence, test, tb_test, scoreboard, Bug injection and verification, Verification plan
- **Janvier:** sequence_item, monitor, driver,Bug injection and verification, Verification plan
- **Sal Esmaeil:** RTL, agent, coverage, Bug injection and verification, Verification Plan

9 Schedule

9.1 Plan of completion.

Table 1: Project Completion Plan

Milestone	Description of Activities	Deliverables	Due	Status
MS1	• Document preparation • structuring files and git • Design implementation and conventional TB • Division of tasks	• Design Spec Doc • Initial Verification Plan • Initial Design Implementation • Conventional Testbench	Jan 30	Completed
MS2 & MS3	• Class-based verification • Defining components: Transaction, Generator, Driver, Monitors, Scoreboard and Coverage • work on code and functional coverage • Debugging error	• Complete RTL • Class-based Testbench • Coverage report • Updated Verification Plan • Report for TB	Feb 18	Completed
MS4	• Develop UVM Testbench • Study UVM architecture, hierarchy & components • Add a section for UVM verification Plan • use UVM_MESSAGING, UVM_LOGGING mechanisms to create and log reports and data.	• Complete RTL • UVM Testbench • Coverage report • Updated Verification Plan • Report for TB	Feb 27	Completed
MS5	• Complete the UVM architecture, UVM environment and UVM testbench • Complete all the testcases and obtain coverage reports. • Create few scenarios of bug-injection and verify. • Preparing presentations and finalize documents.	• Complete UVM Architecture • Coverage report • Finalized Documents • Presentation	Mar 16	Completed

10 Reference uses/Citations/Acknowledgments

10.1 Project GitHub

Github link: `https://github.com/jmpfizi-rutihunza/ece593_g9_final_project`

10.2 Reference

1) Siemens UVM Basics: https://www.youtube.com/watch?v=-WmTiQiTwco&list=PLWPf2kTT1_Z20pIAfQJ2qyNXLCLM

2) Semiconductor Club: https://www.youtube.com/watch?v=JRfmSv5INP8

3) GitHub https://github.com/ahmd-kamel/Memory-Verification-using-UVM/blob/main/Class%20Based%20Env/coverage.s

4) ChipVerify: https://www.chipverify.com/uvm/uvm-testbench-example-1

5) Siemens Verification Academy: https://verificationacademy.com/topics/uvm-universal-verification-methodology/

6) silicondv: https://silicondv.com/topics/popular-articles/uvm-testbench-architecture-a-practical-guide-for-engineers-38

7) IEEE Xplore: https://ieeexplore.ieee.org/document/6906085

8) Research Gate: https://www.researchgate.net/publication/303759959$_{U}VM_{A}RCHITECTURE_{F}OR_{V}ERIFICATIONJ$
$eyJjb250ZXh0Ijp7ImZpcnN0UGFnZSI6Il9kaXJlY3QiLCJwYWdlIjoiX2RpcmVjdCJ9fQ$

10.3 Acknowledgment

we would like to express our sincere gratitude to Professor Venkatesh Patil for their invaluable guidance and technical insights throughout the development of this project. The pre-silicon validation class provided the foundation necessary to architect verification environment using a traditional verification, class-based and UVM architecture. This course helped us understanding the real-industrial verification and challenges.